

EVCalc

Sports Betting Expected Value Calculator

Product	EV Calculator SaaS
Document Type	Software Design Document (SDD)
Status	ACTIVE - Personal Passion Project
Version	1.0.0
Author	Solo Engineer / Owner
Date	April 2026
Frontend	React (Vite) SPA hosted on AWS S3 + CloudFront
Backend	Python (AWS Lambda) REST via AWS API Gateway
Data Source	The Odds API + AWS S3 (odds cache)
Target Users	~1,000 concurrent users (personal project scale)
Purpose	Educated, data-driven, responsible sports betting

React · Python · AWS Lambda · API Gateway · S3 · The Odds API · CloudFront

1. Executive Summary

EdgeCalc is a self-funded passion project designed to bring data-driven decision-making to casual sports bettors. It addresses a fundamental problem in sports betting: most bettors place wagers without knowing whether the odds offered by a sportsbook represent positive or negative expected value relative to the true probability of an outcome.

The application pulls live upcoming soccer match odds from all major US sportsbooks via The Odds API, presents them on a clean event-selection interface, and when the user inputs a wager amount calculates the Expected Value (EV) of that bet across every available sportsbook simultaneously. Results are displayed with immediate visual feedback: positive EV highlighted in green (mathematically favorable), negative EV highlighted in red (mathematically unfavorable).

The system is built on a fully serverless AWS infrastructure for minimal operational cost. Python Lambda functions handle all computation and data management. The React frontend is a static SPA served from S3 via CloudFront. The entire backend scales automatically to support up to 1,000 concurrent users without any manual intervention.

Dimension	Design Decision
Core Value	Real-time EV calculation across all US sportsbooks for upcoming soccer matches
Data Source	The Odds API live odds for all major US sportsbooks (DraftKings, FanDuel, BetMGM, Caesars, etc.)
UX Signal	Green = positive EV (bet is mathematically favorable), Red = negative EV (sportsbook has edge)
Infrastructure	100% serverless AWS: Lambda + API Gateway + S3 zero always-on servers
Scale Target	1,000 concurrent users designed for personal project with room to grow
Mission	Promote educated, responsible gambling by surfacing mathematical edge transparently

2. Problem Statement

2.1 The Information Gap in Sports Betting

Sports betting in the US has exploded since PASPA was overturned in 2018. Dozens of sportsbooks now compete for bettors, each setting their own odds lines. This creates significant price discrepancies between books on the same event and within those discrepancies lies positive expected value for the bettor.

The core problem is that casual bettors have no efficient way to:

- Compare odds across all available sportsbooks simultaneously for a specific event and outcome.
- Determine whether a given set of odds represents positive or negative expected value relative to the implied true probability.
- Know what their bet is worth mathematically before placing it, given a specific stake amount.

2.2 Pain Points

Pain Point	Impact	Pre-EdgeCalc Reality
No cross-book odds comparison tool	HIGH	Manual tab-switching between 5–10 sportsbook apps
No EV calculation for casual bettors	HIGH	EV math done manually or not at all
No visual positive/negative EV signal	HIGH	Bettors cannot distinguish good value from bad
Soccer odds scattered across sources	MEDIUM	No single view of all US books for soccer
Odds change rapidly pre-game	MEDIUM	Stale data leads to missed +EV windows

2.3 The Responsible Gambling Mission

EdgeCalc is built with a specific philosophical intent: making the expected value of every bet visible is an inherently responsible gambling feature. When a bettor sees that 9 out of 10 book odds are red (negative EV), they are empowered to make an informed decision. Transparency about mathematical edge promotes discipline and helps users understand the house advantage built into sportsbook pricing.

3. Expected Value Mathematical Foundation

3.1 What is Expected Value?

Expected Value (EV) is the average outcome of a bet if it were placed an infinite number of times. A **positive EV bet** means that, on average, a bettor will profit over time. A **negative EV bet** means the sportsbook has a mathematical edge.

3.2 EV Formula

Expected Value Formula:

$$EV = (\text{Probability of Win} \times \text{Profit if Win}) - (\text{Probability of Loss} \times \text{Stake})$$

Where:

$$\text{Profit if Win} = \text{Stake} \times (\text{Decimal Odds} - 1)$$

$$\text{Probability of Loss} = 1 - \text{Probability of Win}$$

Example:

Event: Man City vs Arsenal Man City to Win

Sportsbook: DraftKings, Odds: +150 (American) = 2.50 (Decimal)

Stake: \$100

True Prob: 52% (derived from no-vig consensus across all books)

$$\begin{aligned} EV &= (0.52 \times \$150) - (0.48 \times \$100) \\ &= \$78.00 - \$48.00 \\ &= +\$30.00 \rightarrow \text{POSITIVE EV (green)} \end{aligned}$$

Interpretation: On average, this bet earns \$30 per \$100 wagered.

3.3 True Probability from No-Vig Odds

Sportsbooks build a "vig" (vigorish / juice) into their odds the margin that ensures they profit regardless of outcome. EdgeCalc strips the vig using the Pinnacle Sharp Method (a consensus no-vig calculation) to derive the true implied probability of each outcome, which is then used as the basis for EV calculation.

Implied Probability from American Odds:

If odds > 0 (positive): $\text{Implied Prob} = 100 / (\text{odds} + 100)$

If odds < 0 (negative): $\text{Implied Prob} = |\text{odds}| / (|\text{odds}| + 100)$

No-Vig True Probability (for a 2-outcome market):

Raw Prob Home = `implied_prob(home_odds)`

Raw Prob Away = `implied_prob(away_odds)`

Total = Raw Prob Home + Raw Prob Away (will be > 1.0 due to vig)

True Prob Home = Raw Prob Home / Total

True Prob Away = Raw Prob Away / Total

The "true probability" is computed as the average across all available books for that market, providing a consensus sharp-line estimate.

4. System Architecture

4.1 High-Level Architecture

EdgeCalc is a fully serverless, event-driven application on AWS. The React SPA is served statically from S3 via CloudFront. All backend logic runs in AWS Lambda functions invoked through API Gateway. Odds data from The Odds API is fetched by a scheduled Lambda and cached in S3. EV computation happens in a separate Lambda per request.



4.2 AWS Components Summary

Service	Role	Configuration
S3 (frontend)	Static hosting	React build artifacts; public read blocked access via CloudFront OAC only
CloudFront	CDN / Delivery	HTTPS, gzip, edge caching for static assets; WAF association; custom domain
API Gateway (REST)	API Router	2 routes: GET /events, POST /calculate; Lambda proxy integration; usage plans
Lambda: GetEvents	Events list	Python 3.12; 128 MB RAM; 10s timeout; reads S3 odds cache; returns event summaries
Lambda: CalculateEV	EV engine	Python 3.12; 256 MB RAM; 15s timeout; reads S3 odds; computes EV per sportsbook
Lambda: OddsFetcher	Data sync	Python 3.12; 512 MB RAM; 60s timeout; EventBridge 15-min cron; calls Odds API
S3 (odds cache)	Data store	JSON odds files; 24-hour TTL lifecycle policy; versioning disabled; private bucket
EventBridge	Scheduler	rate(15 minutes) rule triggers OddsFetcher Lambda; odds freshness management
Secrets Manager	Secrets	THE_ODDS_API_KEY stored and rotated here; Lambda reads at cold start
CloudWatch	Observability	Lambda logs, API Gateway access logs, custom EV-request metrics, error alarms
WAF (Web ACF)	Security	Attached to CloudFront; AWS Managed Rules + rate limiting 2,000 req/5min per IP

5. Module Design Specifications

5.1 OddsFetcher Lambda Data Ingestion

The OddsFetcher Lambda is the data backbone of EdgeCalc. It runs every 15 minutes via EventBridge and pulls fresh odds for all upcoming week's soccer events from all US sportsbooks via The Odds API. Results are stored in S3 as structured JSON files, keyed by sport, date, and event ID.

Odds API Integration

Endpoint Called:

```
GET https://api.the-odds-api.com/v4/sports/{sport_key}/odds
    ?apiKey={API_KEY}
    &regions=us
    &markets=h2h
    &oddsFormat=american
    &dateFormat=iso
```

Sport Keys (Soccer):

```
soccer_epl, soccer_usa_mls, soccer_spain_la_liga,
soccer_germany_bundesliga, soccer_italy_serie_a,
soccer_france_ligue_one, soccer_uefa_champs_league
```

S3 Storage Schema:

```
Bucket:  edgedcalc-odds-cache
Key:     odds/{sport_key}/{YYYY-MM-DD}/{event_id}.json
```

```
Content: {
  "event_id":    "abc123",
  "sport":       "soccer_epl",
  "home_team":   "Manchester City",
  "away_team":   "Arsenal",
  "commence_time": "2026-04-20T15:00:00Z",
  "bookmakers": [
    {
      "key":    "draftkings",
      "title":  "DraftKings",
      "markets": [{
        "key": "h2h",
        "outcomes": [
          { "name": "Manchester City", "price": 150 },
          { "name": "Arsenal",         "price": -170 },
          { "name": "Draw",            "price": 250 }
        ]
      }]
    },
    ... (all US sportsbooks)
```

5.2 GetEvents Lambda Event List API

Returns the list of upcoming soccer events for the current week, aggregated across all configured soccer leagues. This populates the event-selection dropdown in the React frontend.

API Contract

Request:

```
GET /events

Headers: x-api-key: {user_api_key}
```

Response (200 OK):

```
{
  "events": [
    {
      "event_id": "abc123",
      "sport": "soccer_epl",
      "league_name": "English Premier League",
      "home_team": "Manchester City",
      "away_team": "Arsenal",
      "commence_time": "2026-04-20T15:00:00Z",
      "bookmaker_count": 8,
      "markets": ["h2h"]
    },
    ...
  ],
  "total_events": 24,
  "data_freshness": "2026-04-19T14:00:00Z"
}
```

Lambda Logic:

1. List S3 keys: odds/{today}/*.json and odds/{tomorrow}/*.json
2. For each file, read metadata (no full parse) → build event summary
3. Filter: commence_time within next 7 days
4. Sort: by commence_time ascending
5. Return aggregated events array

5.3 CalculateEV Lambda EV Computation Engine

The core computation Lambda. It receives the selected event ID, chosen outcome (home win / draw / away win), and the user's stake amount. It reads the cached odds from S3, computes the true no-vig probability via consensus across all books, then calculates EV for each sportsbook. Results are annotated with a color signal (GREEN / RED) and returned to the frontend.

API Contract

Request:

```
POST /calculate

Headers: x-api-key: {user_api_key}

Body: {
  "event_id": "abc123",
  "outcome": "Manchester City",    // home | away | draw
  "stake":    100.00                // USD
}
```

Response (200 OK):

```
{
  "event": {
    "home_team": "Manchester City",
    "away_team": "Arsenal",
    "outcome_selected": "Manchester City",
    "commence_time": "2026-04-20T15:00:00Z"
  },
  "true_probability": 0.524,
  "stake": 100.00,
  "results": [
    {
      "sportsbook": "DraftKings",
      "american_odds": 150,
      "decimal_odds": 2.50,
      "implied_prob": 0.400,
      "expected_value": 30.00,
      "profit_if_win": 150.00,
      "signal": "GREEN",
      "ev_percent": "+30.00%"
    },
    {
      "sportsbook": "FanDuel",
      "american_odds": 130,
      "decimal_odds": 2.30,
      "implied_prob": 0.435,
      "expected_value": -5.20,
    }
  ]
}
```

EV Computation Logic (Python)

```
# Python pseudocode CalculateEV Lambda core logic

def compute_ev(event_data, selected_outcome, stake):

    bookmakers = event_data["bookmakers"]

    # Step 1: Collect all implied probabilities for selected outcome
    all_implied_probs = []
    for book in bookmakers:

        outcome = find_outcome(book, selected_outcome)

        if outcome:

            imp = american_to_implied_prob(outcome["price"])

            all_implied_probs.append(imp)

    # Step 2: Compute true probability (no-vig consensus)
    # Use simple average of no-vig probabilities across all books
    no_vig_probs = strip_vig_from_each_book(bookmakers, selected_outcome)
    true_prob = sum(no_vig_probs) / len(no_vig_probs)

    # Step 3: Calculate EV per sportsbook
    results = []
    for book in bookmakers:

        outcome = find_outcome(book, selected_outcome)

        if not outcome: continue

        odds = outcome["price"]

        decimal = american_to_decimal(odds)

        profit_if_win = stake * (decimal - 1)

        implied = american_to_implied_prob(odds)

        ev = (true_prob * profit_if_win) - ((1 - true_prob) * stake)

        ev_pct = (ev / stake) * 100

    results.append({

        "sportsbook": book["title"],

        "american_odds": odds,

        "decimal_odds": decimal,
```


6. Frontend Design (React SPA)

6.1 Application Architecture

The frontend is a React Single Page Application built with Vite for fast builds. It is a lightweight, single-page tool with no routing complexity the entire user workflow fits on one screen. The app is hosted as a static site on S3, served via CloudFront.

6.2 Component Tree

```

<App>

  <Header />                                // Logo, responsible gambling notice

  <EventSelector>                            // Step 1: Choose a game

    <LeagueFilter />                        // Filter by league (EPL, MLS, etc.)

    <EventDropdown />                      // List of upcoming events this week

    <DataFreshnessIndicator />             // "Odds updated 5 min ago"

  </EventSelector>


  <OutcomeSelector>                         // Step 2: Choose outcome

    <OutcomeButton label="Home" />         // e.g. "Man City Win"

    <OutcomeButton label="Draw" />

    <OutcomeButton label="Away" />

  </OutcomeSelector>


  <StakeInput>                             // Step 3: Enter bet amount

    <CurrencyInput />                     // USD, numeric, validates > 0

    <CalculateButton />                   // Triggers POST /calculate

  </StakeInput>


  <EVResultsTable>                         // Step 4: Results

    <TrueProbabilityBadge />               // Consensus true probability

    <BestBookHighlight />                 // Top EV sportsbook callout

    <EVRow                                // Per-sportsbook row

      sportsbook="DraftKings"

      odds="+150"

      ev="+$30.00"

      signal="GREEN" />                  // Green bg if EV > 0, Red if EV < 0

    <EVRow signal="RED" ... />

  </EVResultsTable>


  <ResponsibleGamblingFooter />           // Problem gambling resources

</App>

```

6.3 EV Results Table Visual Design

The EV results table is the primary output surface. Each row represents one sportsbook. The row background is dynamically colored based on EV sign.

Sportsbook	Odds	Implied Prob	EV (on \$100)	Signal
DraftKings	+150	40.0%	+\$30.00	POSITIVE EV
Caesars	+140	41.7%	+\$12.40	POSITIVE EV
FanDuel	+130	43.5%	-\$5.20	NEGATIVE EV
BetMGM	+120	45.5%	-\$19.80	NEGATIVE EV

True Consensus Probability: 52.4% | Best Book: DraftKings (+\$30.00 on \$100 stake)

6.4 Responsible Gambling Integration

EdgeCalc includes a persistent responsible gambling footer with the following elements on every page load:

- A clear disclaimer: "EdgeCalc is an educational tool. Positive EV does not guarantee winning any individual bet."
- The National Problem Gambling Helpline number: 1-800-522-4700.
- A link to ncpgambling.org for additional resources.
- A notice that users should only bet amounts they can afford to lose.

7. End-to-End Data Flow

7.1 Scheduled Odds Refresh Flow

```

EventBridge Scheduler (every 15 minutes)
    |
    ▼
OddsFetcher Lambda invoked
    |
    └─ Read THE_ODDS_API_KEY from AWS Secrets Manager
    |
    └─ For each soccer league key [soccer_epl, soccer_usa_mls, ...]
    |     GET api.the-odds-api.com/v4/sports/{sport_key}/odds
    |         ?apiKey={key}&regions=us&markets=h2h&oddsFormat=american
    |
    └─ Parse response → list of events with bookmaker odds
    |
    └─ For each event:
    |     S3.put_object(
    |         Bucket="edgecalc-odds-cache",
    |         Key=f"odds/{sport}/{date}/{event_id}.json",
    |         Body=json.dumps(event_data),
    |         ContentType="application/json"
    |     )
    |
    └─ Log: events_written, api_quota_remaining
    └─ CloudWatch metric: odds_refresh_success / odds_refresh_failure

```

7.2 User Query Flow Events List

```
User opens EdgeCalc in browser
|
▼
React app loads from CloudFront → S3
|
▼
useEffect() → GET /events (API Gateway)
|
▼
API Gateway → GetEvents Lambda
|
├ S3.list_objects_v2(Prefix="odds/") → get all cached files
├ Filter: commence_time within next 7 days
├ For each matching file: S3.get_object() → parse metadata only
└ Return: sorted events array
|
▼
React: populate EventDropdown with event list
User selects: "Man City vs Arsenal Apr 20, 3:00 PM UTC"
User selects outcome: "Manchester City" (Home Win)
User enters stake: $100
```

7.3 User Query Flow EV Calculation

User clicks "Calculate EV"

|

▼

React: POST /calculate

Body: { event_id: "abc123", outcome: "Manchester City", stake: 100 }

|

▼

API Gateway validates request → invokes CalculateEV Lambda

|

└─ Validate input: event_id exists, stake > 0, outcome is valid

|

└─ S3.get_object(Key="odds/soccer_epl/2026-04-20/abc123.json")

|

└─ Parse bookmakers array

|

└─ For each bookmaker → extract odds for "Manchester City"

|

└─ Compute implied probabilities for all outcomes (each book)

|

└─ Strip vig from each book → derive no-vig probabilities

|

└─ Compute consensus true probability (average across books)

|

└─ For each book:

|

EV = (true_prob × profit_if_win) - ((1 - true_prob) × stake)

|

signal = "GREEN" if EV > 0 else "RED"

|

└─ Sort results by EV descending

|

└─ Return results JSON

|

▼

React: render EVResultsTable

GREEN rows: DraftKings +\$30.00, Caesars +\$12.40

RED rows: FanDuel -\$5.20, BetMGM -\$19.80

8. Security Architecture

8.1 Security Principles

- Least privilege: every Lambda has a scoped IAM role no admin or wildcard policies.
- Zero secrets in code: all API keys in AWS Secrets Manager, injected at Lambda cold start.
- No direct S3 public access: CloudFront OAC (Origin Access Control) is the only path to the frontend S3 bucket.
- API Gateway usage plans: every frontend client uses an API key with rate limits to prevent abuse.
- HTTPS everywhere: CloudFront enforces HTTPS; API Gateway uses TLS 1.2+; Odds API calls use HTTPS.
- WAF protection: CloudFront WAF with AWS Managed Rules blocks OWASP Top 10 attack patterns.

8.2 Security Controls Matrix

Layer	Control	Implementation
Network	AWS WAF	Attached to CloudFront; AWS Managed Rules (Common Rule Set + Known Bad Inputs)
Network	Rate Limiting	WAF rate rule: max 2,000 requests/5 minutes per IP; API Gateway: 1,000 req/s burst
Network	HTTPS Enforcement	CloudFront: HTTPS only, redirect HTTP to HTTPS; HSTS header; TLS 1.2 minimum
Network	CORS	API Gateway CORS configured for CloudFront domain only; no wildcard origins
Storage	S3 Block Public Access	Both S3 buckets: BlockPublicAcls=true, BlockPublicPolicy=true, IgnorePublicAcls=true
Storage	CloudFront OAC	Frontend S3 accessible only via CloudFront Origin Access Control; no direct S3 URL access
Storage	Odds Cache Access	Odds S3 bucket: private; Lambda IAM role is sole principal with s3:GetObject + s3:PutObject
API	API Key Auth	API Gateway usage plans; x-api-key header required; keys rotatable without code change
API	Input Validation	Lambda validates: event_id exists in S3, outcome in [home, away, draw], stake > 0 and < 100,000
Secrets	Secrets Manager	THE_ODDS_API_KEY stored in Secrets Manager; OddsFetcher Lambda reads at cold start via boto3
IAM	Lambda Roles	Three separate IAM roles (one per Lambda) each has only necessary permissions (see 8.3)
IAM	No Admin Access	No Lambda has iam:*, ec2:*, or any service it does not directly use
Monitoring	CloudWatch Alarms	Alarm on: Lambda error rate > 1%, API Gateway 5xx > 5/min, WAF blocked requests spike

8.3 IAM Role Definitions

OddsFetcherLambdaRole:

```
secretsmanager:GetSecretValue → arn:.../THE_ODDS_API_KEY
s3:PutObject                  → arn:...edgecalc-odds-cache/*
logs:CreateLogGroup,
logs:CreateLogStream,
logs:PutLogEvents             → arn:.../OddsFetcher*
```

GetEventsLambdaRole:

```
s3:GetObject                  → arn:...edgecalc-odds-cache/*
s3:ListBucket                 → arn:...edgecalc-odds-cache
logs:CreateLogGroup,
logs:CreateLogStream,
logs:PutLogEvents             → arn:.../GetEvents*
```

CalculateEVLambdaRole:

```
s3:GetObject                  → arn:...edgecalc-odds-cache/*
logs:CreateLogGroup,
logs:CreateLogStream,
logs:PutLogEvents             → arn:.../CalculateEV*
```

NO ROLE HAS: iam:*, ec2:*, rds:*, s3:DeleteObject (immutable cache)

9. Scalability & Availability

9.1 Scalability Design 1,000 Concurrent Users

EdgeCalc's serverless architecture scales automatically to handle 1,000 concurrent users without any configuration changes. AWS Lambda handles concurrent invocations independently; API Gateway manages burst traffic; CloudFront absorbs frontend traffic at the edge.

Component	Scale Model	1,000-User Capacity	Limit / Ceiling
CloudFront CDN	Global edge cache	Unlimited (managed)	No practical limit for static SPA
API Gateway	Managed, auto-scale	1,000 req/s burst	Default: 10,000 req/s (adjustable)
Lambda: GetEvents	Concurrent invocations	1,000 concurrent	Default 1,000 / account (increase free)
Lambda: CalculateEV	Concurrent invocations	1,000 concurrent	Each invocation isolated, ~50ms compute
S3 Odds Cache	Object storage	Unlimited GET throughput	3,500 PUT/5,500 GET per prefix per sec
The Odds API	External dependency	Not user-load-dependent	Decoupled via S3 cache API called only by OddsFetcher

9.2 S3 Cache Key Scalability Design Decision

The most important scalability decision in EdgeCalc is the separation of the Odds API call from user-facing requests. The OddsFetcher Lambda runs on a schedule and writes odds to S3. All user-facing Lambdas read from S3 never from the Odds API directly. This means:

- 1,000 concurrent EV calculation requests all read from S3 in parallel with no Odds API quota consumed.
- Odds API rate limits and latency are completely invisible to end users.
- S3 is effectively infinitely scalable for GET operations at this user count.
- Cold start times for CalculateEV Lambda are bounded by S3 read latency (~20–50ms), not external API latency.

9.3 Availability Design

Failure Scenario	Target SLA	Mitigation
Lambda invocation failure	99.95% (AWS SLA)	Lambda auto-retries on service errors; CloudWatch alarm on error rate

API Gateway unavailability	99.95% (AWS SLA)	Multi-AZ by default; no single point of failure
S3 unavailability	99.99% (AWS SLA)	11-nine durability; cross-AZ redundant; last-known odds served on OddsFetcher failure
The Odds API outage	Degraded (external)	S3 cache serves last-fetched odds; data_freshness indicator shown in UI with age warning
OddsFetcher Lambda failure	Degraded (staleness)	Cached odds remain available; UI shows "Data may be stale refreshed X min ago"
CloudFront outage	99.99% (AWS SLA)	Global PoP redundancy; automatic failover to healthy edge nodes
OVERALL application SLA	99.9% target	Composite of Lambda + API GW + S3 + CloudFront; no database dependency

9.4 Lambda Cold Start Mitigation

For a 1,000-user personal project, Lambda cold starts are acceptable. However, the following strategies minimize cold start impact:

- Python runtime with minimal dependencies (boto3 is built-in to Lambda; only json, decimal are additional imports) keeps cold start under 400ms.
- CalculateEV Lambda uses 256 MB memory allocation more memory = faster CPU = faster cold start and compute.
- Provisioned concurrency can be enabled for CalculateEV Lambda (2 provisioned instances at ~\$3/month) if cold starts become a UX issue.
- API Gateway caching can be enabled for GET /events at a 5-minute TTL to reduce Lambda invocations for repeat event list loads.

10. Performance Targets

Metric	Target (p95)	Strategy
React SPA initial load	< 1.5 s	CloudFront edge cache; Vite code splitting; gzip compression
GET /events response (warm)	< 400 ms	S3 list + metadata reads; Lambda warm execution ~80ms compute
GET /events (cold start)	< 1.5 s	Acceptable; mitigated by API Gateway 5-min cache option
POST /calculate (warm)	< 300 ms	Single S3.get_object + Python EV math; deterministic sub-50ms compute
POST /calculate (cold start)	< 1.2 s	256MB Lambda; minimal imports; S3 read is primary latency driver
Odds data freshness	≤ 15 minutes	EventBridge 15-min cron on OddsFetcher; shown to user in UI
EV table render (React)	< 100 ms	Pure client-side React render of JSON; no additional network calls

11. Cost Model (Personal Project)

EdgeCalc is designed to run at near-zero cost for a personal project with ~1,000 users. Every AWS component is serverless with generous free tiers.

Resource	Est. Monthly Cost	Notes
AWS Lambda	\$0	1M requests/month free; 400,000 GB-sec free well within 1K user scale
API Gateway	~\$1–3	\$3.50/million API calls; at 1K users × 10 calls/day = 300K calls/month
S3 (both buckets)	< \$0.50	Frontend: ~5 MB static; odds cache: ~200 JSON files × 20KB = 4MB; GET requests negligible
CloudFront	\$0	1 TB/month free transfer; 10M HTTPS requests free; easily within personal project range
EventBridge	\$0	Free for scheduled rules; OddsFetcher fires ~2,880 times/month
Secrets Manager	\$0.40	\$0.40/secret/month × 1 secret
CloudWatch	\$0	5 GB log ingestion free; basic alarms free
The Odds API	\$0–79	Free tier: 500 req/month (7 leagues × 2,880 calls = upgrade needed); Starter plan \$79/mo for 100K req
WAF	~\$5	\$5/WebACL/month + \$1/rule/month; minimal for basic protection
TOTAL (no Odds API upgrade)	~\$7/month	With free-tier Odds API key (limited leagues or polling frequency)
TOTAL (with Odds API Starter)	~\$87/month	Full league coverage, 15-min refresh, 1K users served reliably

12. Future Roadmap

Phase	Feature	Description
v1.1	More Sports	Expand beyond soccer to NFL, NBA, MLB, NHL using additional Odds API sport keys.
v1.2	Parlay EV	Multi-leg parlay EV calculator compound probability across 2–4 legs.
v1.3	Historical EV Tracker	Log user's past EV calculations vs actual results track realized edge over time.
v2.0	User Accounts	Optional account creation (AWS Cognito) to save bet history, set bankroll size, Kelly Criterion stake sizing.
v2.1	Kelly Criterion	Alongside EV, show optimal Kelly stake size based on bankroll and true probability.
v2.2	Line Movement Alerts	Email/push notification when a line moves into positive EV territory on a saved event.
v3.0	Arbitrage Finder	Detect cross-book arbitrage opportunities where combined book odds guarantee profit.

13. Appendix

A. Glossary

Term	Definition
Expected Value (EV)	The average mathematical outcome of a bet over many trials. Positive EV = bettor has the edge. Negative EV = sportsbook has the edge.
Vig / Juice	The sportsbook's built-in margin added to odds that ensures they profit regardless of outcome. Also called "the house edge."
No-Vig Probability	The implied probability of an outcome with the sportsbook margin removed representing the "true" estimated probability.
American Odds	Odds format used by US sportsbooks. Positive (e.g. +150) = profit on \$100 stake. Negative (e.g. -170) = stake required to win \$100.
Decimal Odds	Odds format: total return per \$1 staked (including stake). Decimal = (American/100) + 1 for positive odds.
The Odds API	Third-party REST API service that aggregates live betting odds from major US sportsbooks across multiple sports and leagues.
Positive EV	A bet where the expected long-run return is greater than the stake. Shown in GREEN in EdgeCalc. Mathematically favorable.
Negative EV	A bet where the expected long-run return is less than the stake. Shown in RED in EdgeCalc. The sportsbook has the mathematical edge.
Responsible Gambling	The principle of betting within personal financial limits with full awareness of the mathematical probabilities involved.
SPA	Single Page Application web app loaded once, updated dynamically. EdgeCalc's React frontend is a SPA.
OAC	Origin Access Control CloudFront mechanism ensuring S3 buckets are only accessible via CloudFront, not directly.

B. Tech Stack Summary

Category	Technology	Version / Notes
Frontend Framework	React	v18+ hooks, functional components, useEffect for API calls
Frontend Build	Vite	v5 fast HMR dev server; optimized production bundle for S3 deploy
Styling	Tailwind CSS	v3 utility-first; green/red EV color classes via conditional className
Backend Language	Python	3.12 boto3 built-in to Lambda runtime; standard library only for EV math

API Layer	AWS API Gateway	REST API; Lambda proxy integration; usage plans with API key auth
Compute	AWS Lambda	3 functions: OddsFetcher, GetEvents, CalculateEV; Python 3.12 runtime
Data Storage	AWS S3	2 buckets: frontend SPA + odds JSON cache; standard storage class
CDN	AWS CloudFront	OAC for S3; HTTPS; WAF association; edge caching for static assets
Scheduler	AWS EventBridge	rate(15 minutes) triggers OddsFetcher Lambda
Secrets	AWS Secrets Manager	1 secret: THE_ODDS_API_KEY; auto-rotation optional
Odds Data	The Odds API	REST API; v4; soccer sport keys; regions=us; markets=h2h; American odds format
WAF	AWS WAF v2	CloudFront WebACL; AWS Managed Rules + rate-based rule

C. Revision History

Version	Date	Author	Changes
0.1	Feb 2026	Solo Engineer	Initial draft architecture proposal
0.5	Mar 2026	Solo Engineer	EV math section + Lambda designs
0.9	Mar 2026	Solo Engineer	Security + scalability finalized
1.0	Apr 2026	Solo Engineer	Final review; cost model + roadmap added